

Abstraction:

```
abstract class Animal {  
    abstract void sound();  
    void sleep() {  
        System.out.println("Animal is sleeping");  
    }  
}  
  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
class Abstraction {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.sound();  
        d.sleep();  
    }  
}
```

Inheritance:

1.Single Inheritance

```
class Animal {  
    void eat() {  
        System.out.println("Animal is eating");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog is barking");  
    }  
}
```

```
class SingleInheritance {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.eat();  
        d.bark();  
    }  
}
```

2.Muti Level Inheritance

```
class Animal {  
    void eat() {  
        System.out.println("Animal is eating");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog is barking");  
    }  
}  
  
class Puppy extends Dog {  
    void weep() {  
        System.out.println("Puppy is weeping");  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
        Puppy p = new Puppy();  
        p.eat(); // From Animal  
        p.bark(); // From Dog  
        p.weep(); // From Puppy  
    }  
}
```

```
}
```

3. Hierarchical Inheritance

```
class Animal {  
    void eat() {  
        System.out.println("Animal is eating");  
    }  
}
```

```
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog is barking");  
    }  
}
```

```
class Cat extends Animal {  
    void meow() {  
        System.out.println("Cat is meowing");  
    }  
}
```

```
class Main {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        Cat c = new Cat();  
        d.eat();  
        d.bark();  
        c.eat();  
        c.meow();  
    }  
}
```

Polimorphism:

1.Methos Overloading

```
class Calculator {  
    int add(int a, int b) {
```

```
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
}

class MethodOverload {
    public static void main(String[] args) {
        Calculator c = new Calculator();

        System.out.println(c.add(10, 20));
        System.out.println(c.add(10, 20, 30));
    }
}
```

2.Method Overriding

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

class MethodOverride {
    public static void main(String[] args) {
        Animal a = new Dog();
        a.sound();
    }
}
```

```
}
```

1.Copy Constructor

```
class Student {  
    String name;  
    int age;  
    Student(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
    Student(Student s) {  
        this.name = s.name;  
        this.age = s.age;  
    }  
    void display() {  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age);  
    }  
}  
  
class Copyconstructor {  
    public static void main(String[] args) {  
        Student s1 = new Student("John", 20);  
        Student s2 = new Student(s1);  
        s1.display();  
        s2.display();  
    }  
}
```

2.Chain Constructor

```
class Student {  
    String name;  
    int age;  
    Student() {
```

```
        this("John", 20); // Calls parameterized constructor
        System.out.println("Default Constructor");
    }
    Student(String name, int age) {
        this.name = name;
        this.age = age;
        System.out.println("Parameterized Constructor");
    }
}

class ChainConstructor {
    public static void main(String[] args) {
        Student s = new Student();
    }
}
```